

Mini Paper — 2.1 Elements of Computational Thinking

Time: ~30 minutes · Total: 25 marks · Answer all questions.

Mark your work against the mark scheme at the end; aim for ~1 minute per mark.

Questions

Scenario. *FastPark* is a company building software for a multi-storey car park. Drivers enter at a barrier, take a digital ticket, park, and pay at a machine before leaving. A companion mobile app lets drivers search nearby car parks, see live space availability, and reserve a bay in advance. The software must control the entry/exit barriers, track how many spaces are free, calculate charges, and serve the app.

Q1 [3] (AO2). The developers model the car park abstractly in software.

(a) State what is meant by *abstraction*. [1]

(b) Give **one** real-world detail about the car park that would be *removed* in the model, and **one** detail that must be *kept*, explaining why each choice is appropriate. [2]

Q2 [4] (AO2). The companion app must show "spaces free" for each car park.

(a) Identify **two inputs** the availability feature needs and **two outputs** it produces. [2]

(b) State **one precondition** that must be true before a driver can reserve a bay, and explain why it is needed. [2]

Q3 [4] (AO3). Live space-availability data for each car park is requested by many drivers' apps every few seconds.

Explain how **caching** this data on the server could improve performance, and **evaluate one trade-off** of doing so in this scenario. [4]

Q4 [4] (AO2). When a driver pays at the exit machine, the software runs through several steps in a fixed order.

(a) List **four** sub-procedures the payment-and-exit process could be decomposed into, in the correct order. [3]

(b) Explain why the order of two of your steps cannot be swapped. [1]

Q5 [4] (AO2). Describe **two** decision points the exit software must handle, and for **each** give the exact **condition** (a Boolean test) and the outcome of **both** branches. [4]

Q6 [6] (AO3). When a driver opens the app's home screen, the software must (a) fetch the list of nearby car parks, (b) fetch live availability for each, (c) load the user's saved payment cards, and (d) render the map background.

Discuss which of these tasks could be performed **concurrently**, and evaluate the **benefits and trade-offs** of using concurrent processing here. [6]
